

# Lab 2: Super Learner

## Causal Machine Learning

Stijn Vansteelandt and Federico Bertoina, Ghent University

---

We will be using the Student Dropout Prediction Dataset, which simulates a realistic academic environment for 10,000 students, from which we subsample  $n = 2,000$  observations for computational speed. The dataset contains demographic, academic, and socioeconomic variables. We consider two prediction targets: (i) **Dropout**, a binary indicator of whether a student dropped out, and (ii) **GPA**, a continuous measure of academic performance. The goal of this lab is to get familiar with the **SuperLearner** R package and to understand the role of cross-validation and cross-fitting — tools that will be central in subsequent labs on causal inference. For a comprehensive reference on the package's functions and arguments, see the **SuperLearner** documentation.

### 1. Data exploration and preprocessing.

- (a) Load the dataset and inspect its structure using `glimpse()`. Which variables are numeric? Which are categorical?
- (b) Visualise the distribution of the outcome variables. Is **Dropout** balanced? Is **GPA** approximately normally distributed?
- (c) Visualise the dropout rate across categorical variables (e.g. **Department**, **Gender**). Do any groups show notably higher dropout rates?
- (d) Encode categorical variables appropriately: binary variables as 0/1, ordinal variables as ordered integers, and nominal variables (e.g. **Department**) via one-hot encoding using `dummy_cols()`.
- (e) Inspect the correlation of each predictor with **Dropout** and **GPA**. Which variables appear most predictive?

### 2. Super Learner on the full data.

The Super Learner is an ensemble method that combines multiple candidate learners (the *library*) by finding the convex combination that minimises cross-validated risk.

- (a) Use `listWrappers()` to explore the available candidate learners. Build a library containing: `SL.mean`, `SL.glm`, `SL.ranger`, `SL.gam`, `SL.earth`.
- (b) Fit a Super Learner for **Dropout** (classification) and for **GPA** (regression) on the full dataset using `SuperLearner()`.
- (c) Inspect the fitted object. Which learners received the largest ensemble weights? What does a weight of zero imply?
- (d) Compute in-sample predictions and evaluate performance: use MSE and AUC for **Dropout**, and MSE and  $R^2$  for **GPA**. Plot the ROC curve and the predicted vs. observed GPA.
- (e) Do you think in-sample metrics are optimistic? If yes, why?

### 3. Cross-validated risk with `CV.SuperLearner`.

To obtain honest estimates of predictive performance, we evaluate the Super Learner using an outer cross-validation loop via `CV.SuperLearner()`. This results in a nested cross-validation: the outer loop evaluates the ensemble, while the inner loop fits the ensemble weights.

- (a) Run `CV.SuperLearner()` for both outcomes using  $V = 5$  outer folds and  $V = 5$  inner folds (`innerCvControl`).
- (b) Inspect the output using `summary()` and `plot()`. How does cross-validated risk compare across candidate learners and the Super Learner ensemble?

#### 4. Cross-fitting.

Cross-fitting trains the Super Learner on one part of the data and predicts on the held-out part, cycling through all folds. This produces out-of-fold predictions  $\hat{f}^{-k}(X_i)$  for every observation  $i$ , where the model used to predict  $X_i$  was never trained on  $X_i$ .

- (a) Implement cross-fitting manually using a `for` loop over  $K$  folds. For each fold  $k$ : train the Super Learner on the complement  $\{i : \text{fold}(i) \neq k\}$  and predict on fold  $k$ .
- (b) Evaluate the cross-fitted predictions for both outcomes. How do the cross-fitted MSE and AUC compare to the in-sample metrics from Question 2?

#### 5. Tuning base learners with `create.Learner()`.

Individual learners in the library can be tuned over a grid of hyperparameters using `create.Learner()`, which generates one wrapper per hyperparameter configuration.

- (a) Use `create.Learner()` to create tuned variants of `SL.ranger` (varying `mtry` and `num.trees`) and `SL.earth` (varying `degree`).
- (b) Create tuned variants of `SL.xgboost` varying `max_depth`.
- (c) Build an expanded library (`SL.library_tuned`) combining the tuned variants with `SL.mean`, `SL.glm`, and `SL.gam`.
- (d) Re-run `CV.SuperLearner()` with the tuned library for both outcomes. Does tuning improve cross-validated performance relative to Question 3?
- (e) Re-run cross-fitting with the tuned library. Compile a final comparison table of cross-fitted MSE, AUC, and  $R^2$  across both libraries.

#### (bonus) Custom learner: regularised regression.

- (a) Use `create.Learner("SL.glmnet", ...)` to add ridge, elastic net, and lasso to the library.